```
$ docker-machine ls
NAME         ACTIVE DRIVER     STATE   URL        SWARM DOCKER      ERRORS
do-manager-1 -      digitaloc… Running tcp://46…       v18.09.1
do-worker-1  -      digitaloc… Running tcp://13…       v18.09.1
do-worker-2  -      digitaloc… Running tcp://14…       v18.09.1
local-vm-1   *      virtualbox Running tcp://19…       v18.09.1
```

However, at present, these two new instances aren't part of our Swarm cluster. When we initialized our swarm, the output provided the following instructions for adding workers:

```
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0axry1rp0wxy6u48t4epiml4mubf9qy2y2o2f
dmq1u7n2vnj08-aoqoyn55bypasn82emb3sae27 46.101.90.10:2377
```

Let's make our worker instances join the swarm now. For convenience, I suggest setting temporary environment variables in your terminal session for both the swarm token and the manager node's internal IP address:

```
SWARM_TOKEN=SWMTKN-1-0axry1rp0wxy6u48t4epiml4mubf9qy2y2o2fdmq1u7n2vnj08-aoq
oyn55bypasn82emb3sae27
MANAGER_INTERNAL_IP=46.101.90.10
```

Now we can make the workers join the swarm with the commands:

```
$ docker-machine ssh do-worker-1 \
  "docker swarm join --token $SWARM_TOKEN $MANAGER_INTERNAL_IP:2377"
This node joined a swarm as a worker.
```

and:

```
$ docker-machine ssh do-worker-2 \
  "docker swarm join --token $SWARM_TOKEN $MANAGER_INTERNAL_IP:2377"
This node joined a swarm as a worker.
```

If we're using a suitable scripting environment, we could have automated the creation of the worker instances and joining the swarm with a script like the following (using Bash):

```
SWARM_TOKEN=SWMTKN-1-0axry1rp0wxy6u48t4epiml4mubf9qy2y2o2fdmq1u7n2vnj08-aoq
oyn55bypasn82emb3sae27
MANAGER_INTERNAL_IP=46.101.90.10

for i in 1 2
do
  # create the node
  docker-machine create \
    --driver digitalocean \
    --digitalocean-access-token $DIGITAL_OCEAN_TOKEN \
    do-worker-$i
```